



Universidade do Oeste de Santa Catarina
Curso: Ciência da Computação
Componente Curricular: Programação III
Professor: Wagner Titon



Sistema Web e Mobile para Controle e Contagem de Estoque

Integrantes do grupo:
Thauan Gustavo Kerber
Augusto Lima Hagemeier
Gilmar Antes Júnior
João Vitor Tragancin
Gabriel Corso Andrade

Videira - SC
2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. OBJETIVO DA SOLUÇÃO.....	4
3. ESCOPO DO SISTEMA.....	5
4. TECNOLOGIAS UTILIZADAS.....	6
5. ARQUITETURA DO SISTEMA.....	7
6. FUNCIONALIDADES DO SISTEMA.....	9
6.1 Login e autenticação.....	9
6.2 Dashboard.....	10
6.3 Cadastro de produtos.....	10
6.4 Cadastro de categorias.....	10
6.5 Cadastro de fornecedores.....	10
6.6 Entrada e saída de estoque.....	11
6.7 Histórico de movimentações.....	11
6.8 Contagem de estoque.....	11
6.9 Relatório de divergências.....	12
6.10 Aplicativo mobile para contagem.....	12
6.11 API REST.....	12
6.12 Controle de usuários e perfis.....	12
7. DESIGN PATTERNS UTILIZADOS.....	13
7.1 Repository Pattern.....	13
7.2 Service Layer.....	14
7.3 DTO.....	14
7.4 Dependency Injection.....	15
7.5 Facade.....	15
7.6 Resumo da aplicação dos padrões.....	15
8. BANCO DE DADOS.....	16
8.1 Tabelas do sistema.....	16
8.1.1 Tabela users.....	17
8.1.2 Tabela companies.....	17
8.1.3 Tabela categories.....	17
8.1.4 Tabela suppliers.....	18
8.1.5 Tabela products.....	18
8.1.6 Tabela stock_movements.....	19
8.1.7 Tabela inventory_counts.....	19
8.1.8 Tabela inventory_count_items.....	20
8.2 Relacionamentos.....	20
8.3 Estrutura simplificada do banco.....	21
8.4 Controle das movimentações.....	22
8.5 Controle das contagens de estoque.....	22
8.6 Scripts de criação e dados de teste.....	22
9. API REST.....	23
9.1 Objetivo da API.....	23

9.2 Autenticação da API.....	23
9.3 Principais rotas da API.....	24
9.4 Retornos em JSON.....	25
9.5 Validações.....	26
9.6 Tratamento de erros.....	26
9.7 Integração com o aplicativo mobile.....	27
10. APLICATIVO MOBILE.....	27
10.1 Objetivo do aplicativo.....	28
10.2 Tecnologias utilizadas no mobile.....	28
10.3 Funcionamento da contagem pelo celular.....	29
10.4 Uso do código de barras.....	30
10.5 Persistência local com Room Database.....	30
10.6 Integração com a API REST.....	30
10.7 Benefícios do aplicativo mobile.....	31
11. DIAGRAMAS.....	31
11.1 Diagrama de arquitetura.....	31
11.2 Diagrama do banco de dados.....	32
11.3 Diagrama do fluxo de contagem.....	34
11.4 Importância dos diagramas.....	35
12. PRINTS DA APLICAÇÃO.....	35
12.1 Tela de login.....	35
12.2 Dashboard.....	36
12.3 Tela de listagem de produtos.....	36
12.4 Tela de cadastro de produto.....	37
12.5 Tela de categorias.....	38
12.6 Tela de fornecedores.....	38
12.7 Tela de movimentações de estoque.....	39
12.8 Tela de histórico de movimentações.....	39
12.9 Tela de criação de contagem de estoque.....	40
12.10 Tela mobile de contagem de estoque.....	41
12.11 Tela de divergências.....	41
12.12 Tela de aprovação de ajustes.....	42
13. DIFICULDADES ENCONTRADAS.....	42
14. CONCLUSÃO.....	44
15. REFERÊNCIAS.....	44

1. INTRODUÇÃO

O presente documento tem como objetivo apresentar o projeto Counter, uma solução web e mobile voltada para o controle e a contagem de estoque. O sistema foi desenvolvido para atender empresas de diferentes segmentos que necessitam organizar seus produtos, acompanhar movimentações de entrada e saída, registrar contagens físicas e comparar as informações cadastradas no sistema com a quantidade real encontrada no estoque.

O projeto foi desenvolvido utilizando PHP com o framework Laravel, banco de dados MySQL e uma arquitetura organizada em camadas, buscando separar corretamente as responsabilidades do sistema. Além da aplicação web, o Counter conta com um aplicativo mobile Android, desenvolvido em Kotlin, destinado à realização da contagem de estoque por meio do celular. Essa integração entre web e mobile é realizada por meio de uma API REST.

A proposta do Counter é oferecer uma ferramenta simples, funcional e bem estruturada, permitindo que administradores, estoquistas e contadores de estoque trabalhem de forma integrada. A aplicação web é responsável pelo gerenciamento geral do sistema, enquanto o aplicativo mobile é utilizado no processo prático de contagem dos produtos.

Durante o desenvolvimento, foram aplicados conceitos de arquitetura de software, organização em camadas, persistência de dados, API REST e Design Patterns. Dessa forma, o projeto demonstra não apenas o funcionamento de um sistema de controle de estoque, mas também a aplicação de boas práticas de desenvolvimento de software estudadas na disciplina de Programação III.

2. OBJETIVO DA SOLUÇÃO

O objetivo do Counter foi desenvolver uma solução web e mobile para auxiliar empresas no controle, movimentação e contagem de estoque. O sistema centraliza as informações dos produtos, permitindo que os usuários realizem cadastros, registrem entradas e saídas, acompanhem o histórico de movimentações e realizem contagens físicas de forma mais organizada.

A aplicação tem como finalidade reduzir falhas comuns no controle manual de estoque, como divergências entre a quantidade registrada no sistema e a quantidade real disponível na empresa. Para isso, o Counter permite que o administrador gerencie os dados pelo sistema web e que o contador realize a conferência física dos produtos por meio do aplicativo mobile.

Outro objetivo importante do projeto foi integrar a aplicação web com o aplicativo mobile utilizando uma API REST. Dessa forma, o sistema web é responsável pela

administração geral do estoque, enquanto o aplicativo mobile é utilizado como ferramenta de apoio para registrar as quantidades encontradas durante a contagem.

Além da funcionalidade prática, o projeto também teve como objetivo aplicar conceitos estudados na disciplina de Programação III, como arquitetura de software, organização em camadas, banco de dados, API REST e Design Patterns. Assim, o Counter atende tanto à necessidade funcional de um sistema de estoque quanto aos requisitos técnicos propostos para o desenvolvimento da atividade.

3. ESCOPO DO SISTEMA

O escopo do Counter contempla uma aplicação web integrada a um aplicativo mobile, com foco no controle, movimentação e contagem de estoque. O sistema é voltado para uso geral, podendo ser aplicado em empresas de diferentes segmentos que necessitam controlar produtos, fornecedores, entradas, saídas e conferências físicas de estoque.

Na aplicação web, são disponibilizadas funcionalidades administrativas para o gerenciamento do estoque. Entre elas, estão o cadastro de produtos, categorias e fornecedores, o registro de entradas e saídas de produtos, a visualização do histórico de movimentações, o acompanhamento geral por meio de dashboard e a criação de processos de contagem de estoque.

O sistema também conta com controle de usuários, permitindo a utilização de diferentes perfis de acesso. Os perfis definidos para o projeto são: Administrador, Estoquista e Contador. O Administrador possui acesso ao gerenciamento geral do sistema, o Estoquista atua nas movimentações de estoque e o Contador utiliza principalmente o aplicativo mobile para registrar as quantidades encontradas durante a contagem física.

O aplicativo mobile tem como principal função apoiar o processo de contagem de estoque. Por meio dele, o contador acessa uma contagem cadastrada, visualiza os produtos vinculados à contagem, informa a quantidade física encontrada e sincroniza essas informações com o sistema web por meio da API REST desenvolvida no Laravel.

Faz parte do escopo do projeto a implementação de uma estrutura organizada, com banco de dados relacional, autenticação de usuários, arquitetura em camadas e aplicação de Design Patterns. O sistema também contempla o campo de código de barras no cadastro de produtos, permitindo registrar uma identificação adicional dos itens.

Não fizeram parte do escopo inicial do projeto a implementação de alerta de estoque mínimo, emissão de notas fiscais, integração com sistemas externos de venda, controle financeiro completo ou gestão de compras. Essas funcionalidades

são consideradas melhorias futuras e não foram desenvolvidas na versão inicial do Counter.

4. TECNOLOGIAS UTILIZADAS

Para o desenvolvimento do projeto Counter, foram utilizadas tecnologias voltadas para a construção de uma aplicação web completa, integrada a um aplicativo mobile e com persistência de dados em banco relacional. A escolha das tecnologias considerou a necessidade de organização do código, facilidade de manutenção, comunicação entre sistemas e aplicação dos conceitos estudados na disciplina de Programação III.

No back-end, foi utilizada a linguagem PHP com o framework Laravel. O Laravel foi escolhido por oferecer uma estrutura moderna para o desenvolvimento de aplicações web, utilizando o padrão MVC, sistema de rotas, controllers, models, migrations, autenticação, validação e suporte à criação de APIs REST. Dessa forma, o framework contribuiu para a organização do projeto e para a separação das responsabilidades dentro da aplicação.

Para o banco de dados, foi utilizado o MySQL, responsável por armazenar as informações principais do sistema, como usuários, empresas, produtos, categorias, fornecedores, movimentações de estoque, contagens e itens contados. A modelagem foi realizada com relacionamentos entre as tabelas, garantindo a integridade dos dados e permitindo consultas estruturadas.

A interface web foi desenvolvida utilizando Laravel Blade, HTML, Tailwind CSS, Alpine.js e Lucide Icons. O Blade foi utilizado como mecanismo de templates do Laravel, permitindo a criação das telas do sistema de forma integrada ao back-end. O Tailwind CSS foi utilizado para a estilização da interface, possibilitando a construção de telas modernas, responsivas e organizadas por meio de classes utilitárias. O Alpine.js foi utilizado para adicionar interatividade leve na interface, como abertura de menus, modais, filtros e comportamentos dinâmicos sem a necessidade de um framework front-end mais complexo. Já o Lucide Icons foi utilizado para inserir ícones visuais nas telas, contribuindo para uma navegação mais clara e intuitiva.

A comunicação entre a aplicação web e o aplicativo mobile é realizada por meio de uma API REST desenvolvida no Laravel. Essa API permite que o aplicativo mobile consulte dados do sistema, envie informações de contagem e trabalhe de forma integrada com o banco de dados principal da aplicação.

O aplicativo mobile foi desenvolvido em Android nativo com Kotlin, seguindo o modelo apresentado em aula. Para a persistência local dos dados no aplicativo, foi utilizado o Room Database, biblioteca que permite armazenar informações no próprio dispositivo de forma estruturada. Essa abordagem possibilita que o contador

registre informações de contagem no celular e posteriormente envie os dados para a API REST do sistema.

Também foi utilizado o GitHub para versionamento e disponibilização do código-fonte do projeto. O uso do versionamento permitiu melhor organização do desenvolvimento, controle das alterações realizadas e acesso ao projeto pelos integrantes do grupo e pelo professor.

Repositório do projeto: <https://github.com/tawamark/counter-app>

De forma resumida, as principais tecnologias utilizadas no projeto foram:

Back-end: PHP com Laravel

Banco de dados: MySQL

Front-end web: Laravel Blade, HTML, Tailwind CSS, Alpine.js e Lucide Icons

Mobile: Android nativo com Kotlin

Banco local mobile: Room Database

API: REST

Versionamento: Git e GitHub

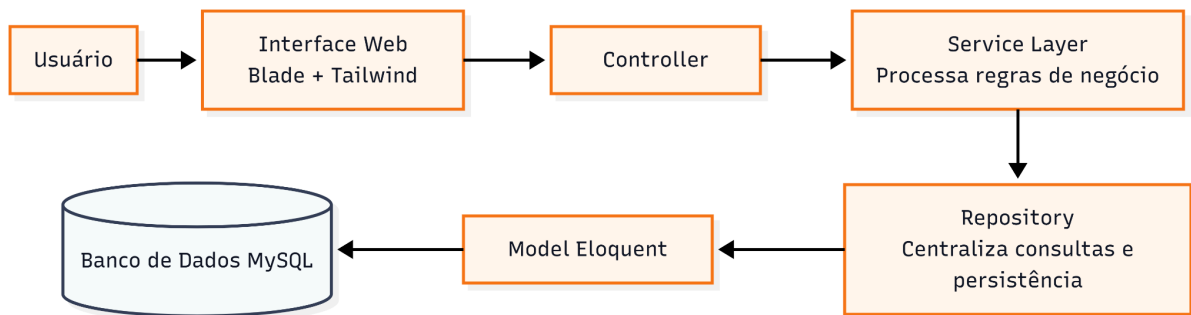
5. ARQUITETURA DO SISTEMA

A arquitetura adotada no projeto Counter é baseada no padrão MVC, utilizado pelo framework Laravel, com a adição de camadas auxiliares para melhorar a organização do código e separar as responsabilidades da aplicação. Além da estrutura tradicional composta por Models, Views e Controllers, o sistema também utiliza camadas de Services, Repositories e DTOs.

O objetivo dessa organização é evitar que as regras de negócio fiquem concentradas diretamente nos Controllers. Dessa forma, cada camada do sistema possui uma função específica, tornando o código mais limpo, reutilizável e fácil de manter.

Na aplicação web, as Views são responsáveis pela interface visual do sistema, desenvolvida com Laravel Blade, Tailwind CSS, Alpine.js e Lucide Icons. Os Controllers recebem as requisições dos usuários, validam o fluxo inicial e encaminham os dados para os Services. Os Services concentram as regras de negócio, como cadastro de produtos, movimentações de estoque, criação de contagens e comparação entre estoque registrado e estoque contado. Os Repositories são responsáveis pela comunicação com o banco de dados, isolando as consultas e operações de persistência. Os Models representam as entidades do sistema e seus relacionamentos com o banco de dados MySQL.

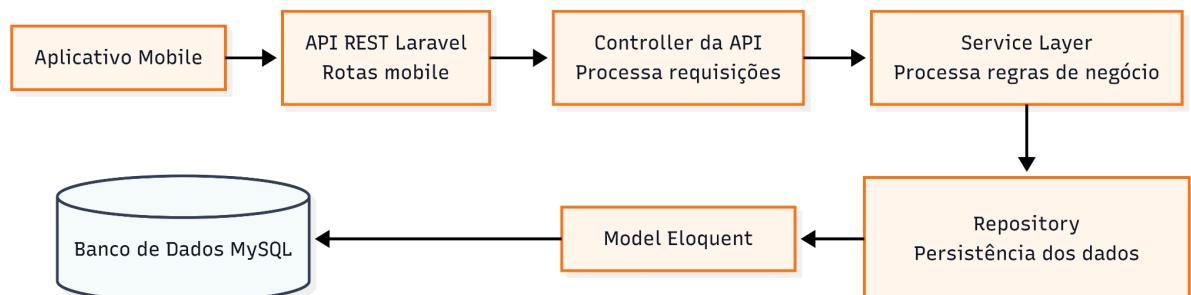
A estrutura geral da arquitetura web pode ser representada da seguinte forma:



Essa separação contribui para uma melhor manutenção do sistema, pois cada parte do projeto possui uma responsabilidade bem definida. Caso seja necessário alterar uma regra de negócio, por exemplo, a alteração pode ser feita na camada de Service, sem impactar diretamente a interface ou a estrutura de acesso ao banco de dados.

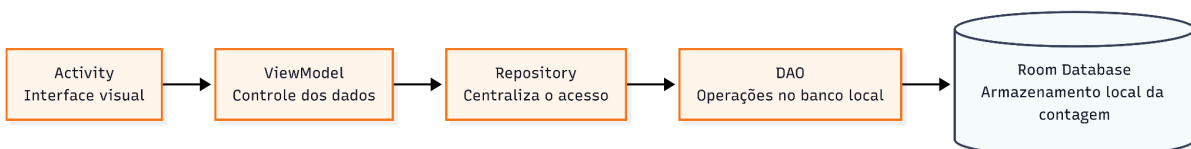
No caso da API REST, a arquitetura segue a mesma lógica. As requisições feitas pelo aplicativo mobile são recebidas por rotas específicas da API, processadas pelos Controllers, encaminhadas para os Services e persistidas no banco de dados por meio dos Repositories. As respostas da API são retornadas em formato JSON.

A estrutura da API pode ser representada da seguinte forma:



O aplicativo mobile foi desenvolvido em Android nativo com Kotlin, seguindo a estrutura apresentada em aula. A arquitetura mobile utiliza Activity, ViewModel, Repository, DAO e Room Database. Essa organização permite separar a interface, a lógica de apresentação, o acesso aos dados e a persistência local no dispositivo.

A estrutura da arquitetura mobile pode ser representada da seguinte forma:



No contexto do Counter, o aplicativo mobile é utilizado principalmente para a contagem física dos produtos. O contador acessa as contagens disponíveis, visualiza os itens da contagem, informa as quantidades encontradas e sincroniza os

dados com a API REST. O Room Database é utilizado para armazenar localmente os dados da contagem, permitindo que as quantidades sejam preservadas antes do envio ao servidor.

Portanto, a arquitetura do Counter integra aplicação web, API REST, banco de dados e aplicativo mobile de forma organizada. Essa estrutura demonstra a aplicação prática de conceitos de arquitetura de software, separação em camadas e boas práticas de desenvolvimento.

6. FUNCIONALIDADES DO SISTEMA

O sistema Counter é composto por funcionalidades voltadas ao gerenciamento completo de estoque, permitindo o cadastro de informações, o controle de movimentações, a realização de contagens físicas e a integração entre a aplicação web e o aplicativo mobile. As funcionalidades foram definidas com o objetivo de atender empresas de diferentes segmentos que necessitam controlar seus produtos de forma organizada.

6.1 Login e autenticação

O sistema conta com uma funcionalidade de login para controlar o acesso dos usuários. Cada usuário informa suas credenciais para acessar a aplicação, garantindo que apenas pessoas autorizadas possam utilizar o sistema.

A autenticação é utilizada tanto na aplicação web quanto na comunicação com o aplicativo mobile. Dessa forma, o sistema identifica qual usuário realizou determinada ação, como cadastro de produto, movimentação de estoque ou registro de contagem.

Além disso, o sistema conta com diferentes perfis de acesso: Administrador, Estoquista e Contador. Esses perfis são utilizados para organizar as permissões de cada tipo de usuário dentro do sistema.

6.2 Dashboard

O dashboard é a tela inicial da aplicação web após o login. Ele tem como objetivo apresentar uma visão geral do estoque e facilitar o acompanhamento das principais informações do sistema.

No dashboard, são exibidos dados como total de produtos cadastrados, quantidade de categorias, quantidade de fornecedores, últimas movimentações realizadas e contagens de estoque em andamento. Essa tela permite que o usuário visualize rapidamente a situação geral do estoque sem precisar acessar cada módulo separadamente.

6.3 Cadastro de produtos

O cadastro de produtos é uma das funcionalidades principais do sistema. Nele, o usuário registra os produtos que fazem parte do estoque da empresa, informando dados como nome, descrição, SKU, código de barras, unidade de medida, preço de custo, preço de venda, quantidade atual, categoria e fornecedor.

O campo de código de barras é utilizado como uma informação adicional de identificação do produto. Essa informação fica registrada no cadastro do item e pode ser disponibilizada pela API para conferência dos dados durante o processo de contagem. Cada produto fica vinculado a uma categoria, a um fornecedor e à empresa responsável pelo estoque.

Também é possível editar informações dos produtos já cadastrados, permitindo manter os dados atualizados conforme a necessidade da empresa.

6.4 Cadastro de categorias

O cadastro de categorias permite organizar os produtos em grupos. Essa funcionalidade facilita a consulta, o filtro e a organização do estoque.

Exemplos de categorias que podem ser cadastradas no sistema são: eletrônicos, materiais de escritório, alimentos, limpeza, ferramentas, roupas, entre outras. Como o Counter é um sistema de uso geral, as categorias podem ser definidas conforme o segmento da empresa que utiliza a aplicação.

6.5 Cadastro de fornecedores

O cadastro de fornecedores é utilizado para registrar as empresas ou pessoas responsáveis pelo fornecimento dos produtos. Cada fornecedor possui informações como nome, CNPJ, telefone, e-mail e endereço.

Esse cadastro contribui para a rastreabilidade dos produtos, permitindo que a empresa saiba de qual fornecedor determinado item foi adquirido. Além disso, o vínculo entre produtos e fornecedores facilita a organização do estoque e futuras consultas.

6.6 Entrada e saída de estoque

O sistema permite registrar movimentações de entrada e saída de estoque. A entrada é utilizada quando novos produtos são adicionados ao estoque, como em casos de compras, recebimentos ou reposições. A saída é utilizada quando produtos são retirados do estoque, como em vendas, perdas, consumo interno ou outras baixas.

Cada movimentação registra informações como produto, quantidade, tipo da movimentação, motivo, usuário responsável e data da operação. Dessa forma, o sistema mantém um histórico confiável das alterações realizadas no estoque.

No caso de saída de estoque, o sistema valida se existe quantidade suficiente disponível antes de permitir a operação. Essa regra evita que o estoque fique com quantidade negativa e contribui para a consistência dos dados armazenados.

6.7 Histórico de movimentações

O histórico de movimentações permite consultar todas as entradas, saídas e ajustes realizados no estoque. Essa funcionalidade é importante para acompanhar as alterações feitas nos produtos e identificar quando, como e por quem determinada movimentação foi registrada.

O histórico é utilizado pelo administrador e pelo estoquista para realizar conferências, verificar inconsistências e acompanhar o comportamento do estoque ao longo do tempo.

6.8 Contagem de estoque

A funcionalidade de contagem de estoque permite que o administrador crie processos de conferência física dos produtos. A contagem é utilizada para comparar a quantidade registrada no sistema com a quantidade real encontrada no estoque da empresa.

O fluxo da contagem é iniciado na aplicação web, onde o administrador cria uma nova contagem e define os produtos que fazem parte dela. Em seguida, o contador acessa essa contagem pelo aplicativo mobile, visualiza os itens vinculados à contagem e informa a quantidade física encontrada para cada produto.

Essa funcionalidade é um dos principais diferenciais do Counter, pois integra o sistema web com o aplicativo mobile e permite que o processo de conferência seja realizado de forma mais prática.

6.9 Relatório de divergências

Após a realização da contagem de estoque, o sistema compara a quantidade registrada no banco de dados com a quantidade informada pelo contador no aplicativo mobile. Quando há diferença entre essas quantidades, é registrada uma divergência.

O relatório de divergências permite ao administrador visualizar quais produtos apresentaram diferença, qual era a quantidade registrada, qual foi a quantidade contada e qual é a diferença encontrada. Com base nessas informações, o administrador aprova os ajustes necessários no estoque.

Essa funcionalidade evita que as alterações sejam feitas automaticamente sem conferência, garantindo maior controle sobre o processo de correção das quantidades.

6.10 Aplicativo mobile para contagem

O aplicativo mobile é utilizado pelo perfil Contador para realizar a contagem física dos produtos diretamente pelo celular. O aplicativo foi desenvolvido em Android nativo com Kotlin e utiliza Room Database para armazenamento local dos dados.

No aplicativo, o contador acessa as contagens disponíveis, visualiza os produtos vinculados à contagem, com informações como nome, SKU, código de barras, unidade de medida, quantidade do sistema e status de sincronização. Também pode buscar itens por nome, SKU ou código de barras, informar a quantidade encontrada e sincronizar os dados com a API REST do sistema.

6.11 API REST

A API REST é responsável pela comunicação entre o sistema web e o aplicativo mobile. Por meio dela, o aplicativo consulta produtos, acessa contagens, registra itens contados e envia as informações para o sistema principal.

As respostas da API são retornadas em formato JSON, facilitando a comunicação entre o Laravel e o aplicativo Android. A API também possui validações e tratamento de erros para garantir que os dados enviados estejam corretos antes de serem processados.

6.12 Controle de usuários e perfis

O sistema conta com três perfis principais de usuário: Administrador, Estoquista e Contador.

O Administrador é responsável pelo gerenciamento geral do sistema, podendo cadastrar usuários, produtos, categorias, fornecedores, criar contagens, visualizar relatórios e aprovar ajustes de estoque.

O Estoquista é responsável por atividades relacionadas ao estoque, como registrar entradas, saídas e consultar movimentações.

O Contador é o usuário responsável por realizar a contagem física dos produtos por meio do aplicativo mobile, informando as quantidades encontradas no estoque.

Essa separação de perfis contribui para a segurança e organização do sistema, permitindo que cada usuário tenha acesso às funcionalidades relacionadas à sua função.

7. DESIGN PATTERNS UTILIZADOS

No desenvolvimento do projeto Counter, foram utilizados Design Patterns com o objetivo de organizar melhor o código, separar responsabilidades e facilitar a manutenção do sistema. Os padrões escolhidos foram definidos considerando as necessidades de um sistema de controle de estoque, que envolve cadastros, movimentações, contagens, comunicação com banco de dados e integração com aplicativo mobile.

Os principais Design Patterns utilizados no projeto são: Repository Pattern, Service Layer, DTO, Dependency Injection e Facade.

7.1 Repository Pattern

O Repository Pattern foi utilizado para separar a lógica de acesso ao banco de dados das demais camadas do sistema. Em vez de os Controllers ou Services acessarem diretamente os Models para realizar consultas, cadastros, edições ou exclusões, essas operações são centralizadas em classes Repository.

No Counter, esse padrão foi aplicado em classes como ProductRepository, StockMovementRepository e InventoryCountRepository. Cada Repository é responsável por lidar com operações de persistência e consulta relacionadas à sua respectiva área do sistema.

O ProductRepository centraliza consultas de produtos, permitindo organizar o acesso aos dados utilizados no cadastro, listagem e consulta de itens. O StockMovementRepository centraliza operações relacionadas ao histórico de movimentações de estoque. Já o InventoryCountRepository organiza consultas e persistência relacionadas às contagens de estoque.

A utilização desse padrão torna o código mais organizado, pois isola as consultas ao banco de dados em uma camada específica. Além disso, facilita futuras manutenções, pois alterações relacionadas à forma de busca ou persistência dos dados podem ser feitas no Repository sem afetar diretamente os Controllers ou Services.

7.2 Service Layer

O padrão Service Layer foi utilizado para concentrar as regras de negócio do sistema em uma camada específica. Essa abordagem evita que os Controllers fiquem responsáveis por processamentos complexos, mantendo-os com a função principal de receber requisições e direcionar os dados para a camada correta.

No Counter, os Services são responsáveis por regras como registrar entradas, saídas e ajustes de estoque, validar se há quantidade suficiente para uma saída,

criar contagens de estoque, finalizar contagens, aprovar ajustes e registrar operações importantes para auditoria.

Exemplos de classes que representam essa camada são `StockMovementService`, `InventoryCountService` e `AuditLogService`.

O `StockMovementService` processa entradas, saídas e ajustes de estoque, atualizando a quantidade atual do produto e registrando o histórico da movimentação. O `InventoryCountService` controla a criação, finalização e aprovação de contagens de estoque. Já o `AuditLogService` registra operações importantes para auditoria do sistema.

Um exemplo prático de uso do Service Layer ocorre no registro de saída de estoque. Antes de concluir a movimentação, o `StockMovementService` verifica se existe quantidade suficiente disponível. Caso não exista estoque suficiente, a operação não é concluída. Dessa forma, a regra de negócio fica centralizada no Service e não espalhada pelo Controller.

A utilização do Service Layer contribui para um código mais limpo, reutilizável e fácil de testar, pois as principais regras do sistema ficam agrupadas em classes específicas.

7.3 DTO

O padrão DTO, ou Data Transfer Object, foi utilizado para transportar dados entre as camadas do sistema de forma organizada. Em vez de passar arrays ou dados soltos entre Controllers, Services e Repositories, o sistema utiliza objetos específicos para representar os dados recebidos em determinadas operações.

No Counter, os DTOs são utilizados para transportar dados validados entre Controllers, Services e Repositories. Exemplos de DTOs utilizados no projeto são `ProductData`, `StockMovementData` e `InventoryCountData`. O `ProductData` representa os dados de cadastro e edição de produtos, o `StockMovementData` representa os dados de uma movimentação de estoque e o `InventoryCountData` representa os dados necessários para criação de uma contagem.

A utilização de DTOs melhora a organização dos dados trafegados dentro do sistema, torna o código mais previsível e reduz a dependência direta das requisições HTTP dentro das regras de negócio. Assim, os Services recebem dados já estruturados e podem processá-los de maneira mais clara.

7.4 Dependency Injection

O padrão Dependency Injection foi utilizado para reduzir o acoplamento entre as classes do sistema. No Laravel, esse padrão é aplicado por meio do container de

serviços, permitindo que Controllers e Services recebam suas dependências automaticamente.

No Counter, Controllers recebem Services e Repositories por parâmetro, enquanto Services recebem outras classes necessárias pelo construtor. Isso evita que as classes criem manualmente suas dependências, deixando o código mais organizado, testável e fácil de manter.

Esse padrão contribui para a separação de responsabilidades, pois cada classe depende de contratos ou classes específicas para executar sua função, sem precisar conhecer detalhes de criação dessas dependências.

7.5 Facade

O padrão Facade foi utilizado por meio de recursos do Laravel, como a classe DB para controle de transações. Esse padrão fornece uma interface simples para operações mais complexas do framework.

No Counter, o uso de DB::transaction permite garantir que operações críticas, como movimentações de estoque e aprovação de contagens, sejam executadas de forma segura. Caso alguma etapa falhe, a transação é desfeita, evitando inconsistências no banco de dados.

Esse padrão foi escolhido porque facilita o uso de funcionalidades internas do Laravel e ajuda a manter a integridade das informações do estoque.

7.6 Resumo da aplicação dos padrões

De forma resumida, os Design Patterns utilizados no Counter são aplicados da seguinte maneira:

Repository Pattern: organiza o acesso ao banco de dados e centraliza as operações de persistência das entidades do sistema, como produtos, categorias, fornecedores, movimentações e contagens de estoque.

Service Layer: concentra as regras de negócio do sistema, como movimentações de estoque, validações, criação de contagens, comparação de quantidades e processamento de divergências.

DTO: transporta dados entre as camadas da aplicação de forma estruturada e padronizada, evitando o uso de dados soltos ou arrays diretamente nas regras de negócio.

Dependency Injection: reduz o acoplamento entre as classes do sistema, permitindo que Controllers e Services recebam suas dependências de forma automática pelo container de serviços do Laravel.

Facade: simplifica o acesso a funcionalidades internas do Laravel, como o uso de `DB::transaction` em operações críticas de movimentação de estoque e aprovação de ajustes.

Com a aplicação desses padrões, o projeto Counter apresenta uma estrutura mais organizada, modular, reutilizável e alinhada com boas práticas de desenvolvimento de software.

8. BANCO DE DADOS

O banco de dados do projeto Counter foi desenvolvido utilizando MySQL, com o objetivo de armazenar todas as informações necessárias para o funcionamento do sistema web e da integração com o aplicativo mobile. A estrutura do banco foi planejada para permitir o controle de usuários, empresas, produtos, categorias, fornecedores, movimentações de estoque e contagens físicas.

A modelagem do banco segue uma estrutura relacional, utilizando chaves primárias e chaves estrangeiras para representar os vínculos entre as entidades. No Laravel, a criação das tabelas foi realizada por meio de migrations, permitindo versionar a estrutura do banco de dados juntamente com o código-fonte do projeto.

8.1 Tabelas do sistema

As principais tabelas utilizadas no projeto são:

users
companies
categories
suppliers
products
stock_movements
inventory_counts
inventory_count_items
audit_logs
personal_access_tokens

Cada tabela possui uma responsabilidade específica dentro do sistema, contribuindo para a organização dos dados e para a separação das informações de cada módulo. As tabelas `audit_logs` e `personal_access_tokens` também fazem parte da estrutura do projeto, sendo utilizadas, respectivamente, para auditoria das operações e autenticação por token.

8.1.1 Tabela users

A tabela users é responsável por armazenar os usuários que têm acesso ao sistema. Cada usuário possui um perfil de acesso, como Administrador, Estoquista ou Contador.

Campos principais:

id
name
email
password
role
company_id
created_at
updated_at

O campo role é utilizado para identificar o tipo de usuário e controlar suas permissões dentro do sistema. O campo company_id permite vincular o usuário a uma empresa cadastrada.

8.1.2 Tabela companies

A tabela companies é responsável por armazenar as empresas que utilizam o sistema Counter. Mesmo sendo um sistema de uso geral, essa tabela permite organizar os dados por empresa, possibilitando que produtos, usuários, fornecedores e movimentações fiquem vinculados a uma organização específica.

Campos principais:

id
name
cnpj
email
phone
created_at
updated_at

8.1.3 Tabela categories

A tabela categories é utilizada para organizar os produtos em categorias. Essa estrutura facilita a busca, a filtragem e a organização dos itens cadastrados no estoque.

Campos principais:

id
company_id
name
description
created_at
updated_at

Cada categoria está vinculada a uma empresa por meio do campo company_id.

8.1.4 Tabela suppliers

A tabela suppliers é responsável por armazenar os fornecedores dos produtos. Essa tabela permite identificar a origem dos itens cadastrados no estoque.

Campos principais:

id
company_id
name
cnpj
email
phone
address
created_at
updated_at

Cada fornecedor está vinculado a uma empresa.

8.1.5 Tabela products

A tabela products é uma das principais tabelas do sistema, pois armazena os produtos controlados no estoque. Cada produto pode estar vinculado a uma empresa, categoria e fornecedor.

Campos principais:

id
company_id
category_id
supplier_id
name
description
sku
barcode
unit
current_quantity

cost_price
sale_price
created_at
updated_at

O campo sku é utilizado como código interno do produto. O campo barcode armazena o código de barras como informação adicional de identificação do item. O campo current_quantity representa a quantidade atual registrada no sistema. Essa quantidade é alterada por movimentações de estoque e por ajustes aprovados em contagens.

8.1.6 Tabela stock_movements

A tabela stock_movements é responsável por armazenar o histórico de movimentações do estoque. Toda entrada, saída ou ajuste realizado no sistema é registrado nessa tabela.

Campos principais:

id
company_id
product_id
user_id
inventory_count_id
type
quantity
quantity_before
quantity_after
reason
created_at
updated_at

O campo type indica o tipo da movimentação, podendo representar entrada, saída ou ajuste. O campo quantity armazena a quantidade movimentada, enquanto quantity_before e quantity_after registram a quantidade do produto antes e depois da movimentação. O campo inventory_count_id permite vincular uma movimentação a uma contagem de estoque, quando o ajuste tiver origem em uma contagem aprovada.

8.1.7 Tabela inventory_counts

A tabela inventory_counts é responsável por armazenar os processos de contagem de estoque criados no sistema. Cada contagem representa uma conferência física dos produtos.

Campos principais:

id
company_id
created_by
title
status
started_at
finished_at
approved_at
created_at
updated_at

O campo status é utilizado para indicar a situação da contagem, como aberta, em andamento, finalizada ou aprovada. O campo created_by identifica o usuário responsável pela criação da contagem, enquanto approved_at registra a data e hora em que a contagem foi aprovada.

8.1.8 Tabela inventory_count_items

A tabela inventory_count_items é responsável por armazenar os produtos contados dentro de uma contagem de estoque. Essa tabela permite comparar a quantidade registrada no sistema com a quantidade física informada pelo contador.

Campos principais:

id
inventory_count_id
product_id
system_quantity
counted_quantity
difference
counted_by
sync_status
counted_at
created_at
updated_at

O campo system_quantity armazena a quantidade registrada no sistema no momento da contagem. O campo counted_quantity armazena a quantidade física informada pelo contador, e o campo difference registra a diferença entre esses valores. O campo counted_by identifica o usuário que realizou a contagem, counted_at registra o momento da contagem e sync_status indica a situação de sincronização do item com o aplicativo mobile.

8.2 Relacionamentos

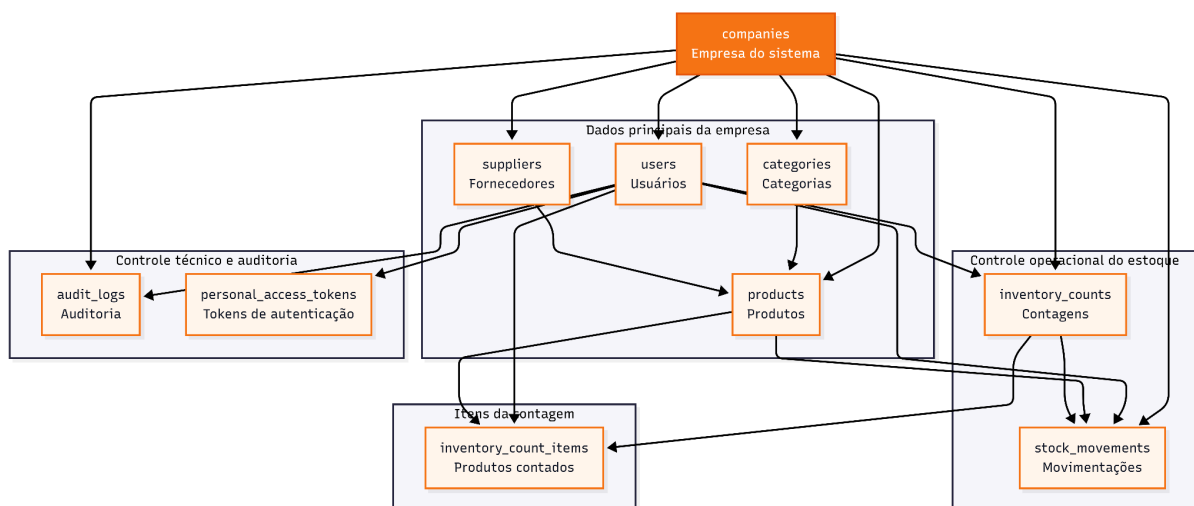
A estrutura do banco de dados do Counter é baseada nos seguintes relacionamentos:

- Uma empresa pode possuir vários usuários.
- Uma empresa pode possuir várias categorias.
- Uma empresa pode possuir vários fornecedores.
- Uma empresa pode possuir vários produtos.
- Uma categoria pode possuir vários produtos.
- Um fornecedor pode fornecer vários produtos.
- Um produto pode possuir várias movimentações de estoque.
- Um usuário pode registrar várias movimentações de estoque.
- Uma empresa pode possuir várias contagens de estoque.
- Uma contagem de estoque pode possuir vários itens contados.
- Um produto pode estar presente em vários itens de contagem.

Esses relacionamentos permitem organizar o sistema de forma consistente, evitando duplicidade desnecessária de dados e garantindo melhor integridade das informações.

8.3 Estrutura simplificada do banco

A estrutura simplificada do banco de dados pode ser visualizada no diagrama a seguir:



Essa estrutura mostra que a empresa é a entidade central do sistema, sendo responsável por agrupar usuários, categorias, fornecedores, produtos, movimentações e contagens de estoque. Além disso, os produtos se relacionam diretamente com categorias, fornecedores, movimentações de estoque e itens de contagem, permitindo que o sistema mantenha a organização e a rastreabilidade das operações realizadas.

8.4 Controle das movimentações

O controle de estoque é realizado principalmente por meio da tabela `stock_movements`. Sempre que ocorre uma entrada, saída ou ajuste de estoque, é criado um registro nessa tabela. Além disso, a quantidade atual do produto é atualizada na tabela `products`.

No caso de entrada de estoque, a quantidade movimentada é somada ao estoque atual do produto. No caso de saída, a quantidade é subtraída, desde que exista saldo suficiente. No caso de ajuste, a quantidade pode ser corrigida a partir de uma contagem física aprovada pelo administrador.

Esse controle permite que o sistema mantenha tanto a quantidade atual do produto quanto o histórico completo das operações realizadas.

8.5 Controle das contagens de estoque

O processo de contagem de estoque é dividido entre as tabelas `inventory_counts` e `inventory_count_items`. A tabela `inventory_counts` armazena a contagem como um processo geral, contendo título, status, empresa, usuário responsável, data de início e data de finalização.

Já a tabela `inventory_count_items` armazena os produtos que fazem parte da contagem, registrando a quantidade existente no sistema, a quantidade informada pelo contador e a diferença encontrada.

Após a finalização da contagem, o sistema apresenta as divergências encontradas. O administrador analisa essas diferenças e aprova os ajustes necessários para atualizar o estoque.

8.6 Scripts de criação e dados de teste

Os scripts de criação do banco de dados foram representados pelas migrations do Laravel. Cada tabela possui sua própria migration, contendo os campos, tipos de dados, chaves primárias, chaves estrangeiras e regras necessárias para a criação da estrutura no MySQL.

Além disso, foram utilizados seeders para inserir dados de teste no sistema, como usuários, empresas, categorias, fornecedores, produtos e movimentações iniciais. Esses dados foram importantes para demonstrar o funcionamento do sistema durante a apresentação final.

Exemplos de dados de teste:

Empresa: Counter Demo

Categorias: Eletrônicos, Materiais de Escritório, Limpeza e Ferramentas

Fornecedores: Tech Distribuidora, Office Supply e Global Atacadista
Produtos: Mouse USB, Teclado Mecânico, Cabo HDMI, Papel A4 e Álcool 70%

Com essa estrutura, o banco de dados do Counter atende às principais necessidades do projeto, permitindo persistência de dados, relacionamentos entre entidades, histórico de movimentações e suporte ao processo de contagem de estoque integrado ao aplicativo mobile.

9. API REST

O projeto Counter conta com uma API REST desenvolvida no Laravel, responsável por permitir a comunicação entre a aplicação web, o banco de dados principal e o aplicativo mobile Android.

A API REST do Counter foi criada para permitir a comunicação entre o aplicativo mobile e o backend web. Ela é utilizada principalmente no processo de contagem de estoque, permitindo que o contador autentique seu acesso, consulte informações necessárias para a contagem, visualize contagens disponíveis, acesse os itens vinculados a cada contagem e sincronize as quantidades informadas no aplicativo.

A utilização da API REST torna o sistema mais flexível, pois separa a comunicação entre o aplicativo mobile e o back-end. Dessa forma, o aplicativo Android consome os dados do sistema por meio de rotas organizadas, com envio e retorno de informações em formato JSON.

9.1 Objetivo da API

O principal objetivo da API é permitir a integração entre o aplicativo mobile e o backend web do Counter. Enquanto a aplicação web é utilizada para o gerenciamento geral do estoque, o aplicativo mobile é utilizado para apoiar a contagem física dos produtos.

Por meio da API, o aplicativo autentica o usuário, consulta produtos, acessa o resumo mobile, lista contagens disponíveis, consulta os itens de uma contagem e envia as quantidades informadas pelo contador para o sistema web.

Dessa forma, a API não tem como foco expor todas as funcionalidades administrativas do sistema, mas sim atender às operações necessárias para a integração entre o aplicativo mobile e o backend.

9.2 Autenticação da API

A API utiliza autenticação para garantir que apenas usuários autorizados acessem as rotas protegidas. O login é realizado por meio de credenciais de usuário, como e-mail e senha. Após a autenticação, o sistema retorna um token de acesso, que deve ser enviado nas próximas requisições feitas pelo aplicativo mobile.

Esse processo permite identificar qual usuário está utilizando o aplicativo e quais ações ele realizou dentro do sistema. Dessa forma, operações como registro de contagem e envio de quantidades ficam vinculadas ao usuário responsável.

Exemplo de rota de autenticação:

POST /api/login

Essa rota recebe os dados de login do usuário e retorna uma resposta contendo as informações básicas do usuário autenticado e o token de acesso.

9.3 Principais rotas da API

As rotas da API são organizadas para atender principalmente à comunicação entre o aplicativo mobile e o backend web. Abaixo estão as principais rotas utilizadas no projeto:

Autenticação:

POST /api/login

POST /api/logout

GET /api/me

Produtos:

GET /api/products/search

GET /api/products

GET /api/products/{product}

Resumo mobile:

GET /api/mobile/summary

Contagens de estoque:

GET /api/inventory-counts

GET /api/inventory-counts/{inventoryCount}

Itens da contagem:

GET /api/inventory-counts/{inventoryCount}/items

POST /api/inventory-counts/{inventoryCount}/items

Sincronização da contagem:

POST /api/inventory-counts/{inventoryCount}/sync

Essas rotas permitem que o aplicativo mobile autentique o usuário, consulte produtos, acesse o resumo mobile, liste contagens disponíveis, visualize os itens de uma contagem e sincronize as quantidades informadas pelo contador com o backend web.

9.4 Retornos em JSON

Todas as respostas da API são retornadas em formato JSON, facilitando a comunicação entre o back-end Laravel e o aplicativo Android.

Esse formato permite que os dados sejam transmitidos de forma estruturada e facilmente interpretados pelo aplicativo mobile.

Exemplo de retorno ao consultar um produto:

```
{
  "id": 1,
  "name": "Mouse USB",
  "sku": "MOU-001",
  "barcode": "7891234567890",
  "current_quantity": 25,
  "unit": "unidade",
  "cost_price": 25.00,
  "sale_price": 49.90
}
```

Exemplo de retorno ao registrar um item contado:

```
{
  "message": "Item contado registrado com sucesso.",
  "data": {
    "product_id": 1,
    "system_quantity": 25,
    "counted_quantity": 23,
    "difference": -2
  }
}
```

9.5 Validações

A API possui validações para garantir que os dados enviados estejam corretos antes de serem processados pelo sistema. Essas validações são importantes para evitar inconsistências no banco de dados e impedir operações inválidas.

Exemplos de validações:

O nome do produto é obrigatório.
O SKU não pode ser duplicado para produtos da mesma empresa.
O código de barras é armazenado como informação adicional de identificação e possui índice para otimizar consultas.
A quantidade movimentada deve ser maior que zero.
Uma saída de estoque não pode ser maior que a quantidade disponível.
Uma contagem só pode ser finalizada se possuir itens contados.
O usuário precisa estar autenticado para acessar rotas protegidas.

Caso uma validação não seja atendida, a API retorna uma mensagem de erro em JSON, informando o problema encontrado.

Exemplo de erro de validação:

```
{
  "message": "Erro de validação.",
  "errors": {
    "quantity": [
      "A quantidade deve ser maior que zero."
    ]
  }
}
```

9.6 Tratamento de erros

A API também conta com tratamento de erros para lidar com situações inesperadas ou inválidas durante a execução das operações. Entre os erros tratados, estão dados não encontrados, acesso não autorizado, erro de validação e falhas internas no processamento.

Exemplos de respostas de erro:

401 Unauthorized: quando o usuário não está autenticado.
403 Forbidden: quando o usuário não possui permissão para determinada ação.
404 Not Found: quando um produto, fornecedor, categoria ou contagem não é encontrado.
422 Unprocessable Entity: quando os dados enviados não passam pelas validações.
500 Internal Server Error: quando ocorre uma falha interna no servidor.

Esse tratamento contribui para tornar a comunicação entre o aplicativo mobile e o sistema web mais segura, previsível e organizada.

9.7 Integração com o aplicativo mobile

A API é utilizada diretamente pelo aplicativo mobile Android no processo de contagem de estoque. O contador acessa o aplicativo, realiza login, consulta as

contagens disponíveis, seleciona uma contagem, visualiza os itens vinculados a ela e pode utilizar busca textual por nome, SKU ou código de barras para localizar os produtos da contagem.

Após o registro das quantidades no aplicativo, os dados são sincronizados com a API, que armazena as informações no banco de dados principal. Posteriormente, o sistema web compara a quantidade registrada no estoque com a quantidade contada pelo usuário no aplicativo.

Fluxo simplificado da integração:

1. O contador realiza login no aplicativo mobile.
2. O aplicativo recebe o token de autenticação.
3. O aplicativo consulta as contagens disponíveis pela API.
4. O contador seleciona uma contagem.
5. O aplicativo carrega os itens vinculados à contagem.
6. O contador informa as quantidades encontradas.
7. O aplicativo sincroniza os dados com a API.
8. O sistema web processa as informações e calcula as divergências.

Com essa API, o Counter garante uma integração organizada entre a aplicação web e o aplicativo mobile, permitindo que o processo de contagem seja realizado de forma prática e conectado ao banco de dados principal do sistema.

10. APLICATIVO MOBILE

O projeto Counter conta com um aplicativo mobile Android desenvolvido em Kotlin, com o objetivo de auxiliar no processo de contagem física de estoque. O aplicativo é utilizado principalmente pelo usuário com perfil Contador, permitindo que ele realize a conferência dos produtos diretamente pelo celular.

A inclusão do aplicativo mobile torna o processo de contagem mais prático e eficiente. Em vez de utilizar planilhas, papéis ou anotações manuais, o contador acessa uma contagem aberta, visualiza os produtos vinculados a ela e informa as quantidades encontradas fisicamente no estoque.

10.1 Objetivo do aplicativo

O principal objetivo do aplicativo mobile é permitir que o contador registre a quantidade física encontrada nos produtos durante uma contagem de estoque. Essas informações são sincronizadas com o sistema web, onde são comparadas com a quantidade registrada no banco de dados.

O aplicativo é utilizado como uma extensão do sistema web. Enquanto a aplicação principal é responsável pelo gerenciamento do estoque, cadastro de produtos,

movimentações, criação de contagens e análise das divergências, o aplicativo mobile é focado na etapa prática da contagem física.

Dessa forma, o Counter oferece uma solução mais completa, integrando o ambiente administrativo web com o processo físico realizado no estoque, sem depender de planilhas ou registros manuais para informar as quantidades encontradas.

10.2 Tecnologias utilizadas no mobile

O aplicativo mobile foi desenvolvido em Android nativo utilizando a linguagem Kotlin, seguindo o modelo apresentado em aula. A estrutura do aplicativo foi organizada com separação de responsabilidades, utilizando classes específicas para interface, controle de dados, acesso ao banco local e comunicação com os dados da aplicação.

Para a persistência local dos dados, foi utilizado o Room Database. O Room permite armazenar dados no próprio dispositivo móvel de forma estruturada, sendo utilizado para guardar informações relacionadas à contagem, produtos e itens contados.

A arquitetura mobile é composta principalmente por:

Activity: responsável pela interface visual e interação com o usuário.

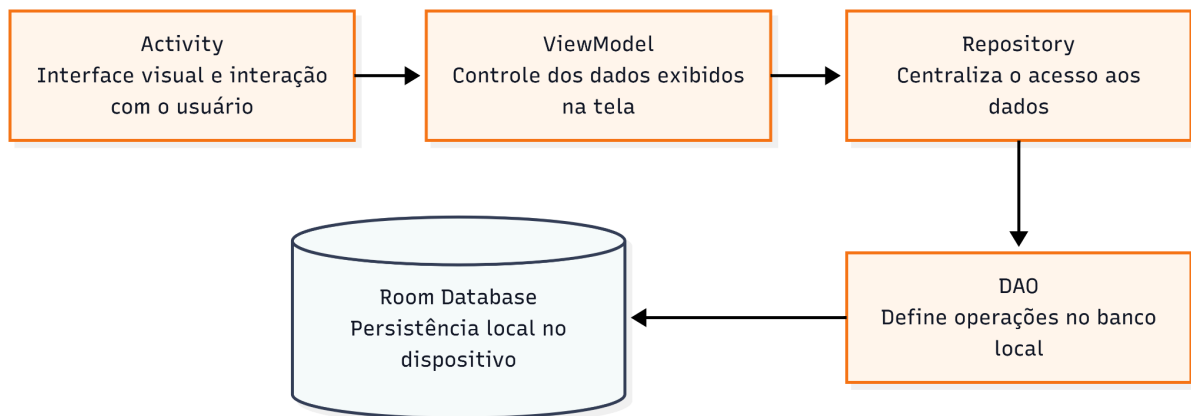
ViewModel: responsável por controlar os dados exibidos na tela e manter a lógica de apresentação.

Repository: responsável por centralizar o acesso aos dados, podendo buscar informações tanto localmente quanto por meio da API.

DAO: responsável por definir as operações realizadas no banco local, como inserir, listar, atualizar e buscar registros.

Room Database: responsável por armazenar os dados localmente no dispositivo Android.

A estrutura simplificada do aplicativo mobile pode ser representada conforme o diagrama a seguir:



Essa organização contribui para um aplicativo mais limpo, estruturado e de fácil manutenção.

10.3 Funcionamento da contagem pelo celular

O fluxo de uso do aplicativo mobile é iniciado quando o contador realiza login no sistema. Após a autenticação, o aplicativo consulta as contagens disponíveis e permite que o usuário selecione a contagem que será realizada.

Depois de acessar uma contagem, o contador visualiza os produtos vinculados a ela, podendo utilizar busca textual por nome, SKU ou código de barras. Em seguida, informa a quantidade física encontrada no estoque.

O fluxo básico do aplicativo é:

1. O contador acessa o aplicativo mobile.
2. O contador realiza login com suas credenciais.
3. O aplicativo consulta as contagens disponíveis.
4. O contador seleciona uma contagem de estoque.
5. O aplicativo exibe os produtos vinculados à contagem.
6. O contador visualiza as informações dos itens da contagem.
7. O contador informa a quantidade física encontrada.
8. O aplicativo salva a informação localmente.
9. O aplicativo sincroniza os dados com a API REST.
10. O sistema web processa os dados e calcula as divergências.

Esse fluxo permite que a contagem seja realizada de maneira organizada, com registro digital das quantidades encontradas.

10.4 Uso do código de barras

O cadastro de produtos do Counter possui um campo específico para código de barras, permitindo registrar uma identificação adicional do item. Essa informação é

disponibilizada pela API e pode ser usada para conferência dos produtos durante a contagem.

Nesta versão do aplicativo mobile, não foi implementada leitura por câmera. A localização dos itens é feita por busca textual, permitindo pesquisar por nome, SKU ou código de barras.

Essa abordagem mantém o sistema próximo de um cenário real de controle de estoque, onde muitos produtos possuem identificação por código de barras, mas preserva a fidelidade entre a documentação e o que foi efetivamente implementado no projeto.

10.5 Persistência local com Room Database

O uso do Room Database no aplicativo mobile permite armazenar localmente informações importantes para o processo de contagem. Isso inclui dados das contagens, produtos vinculados à contagem, itens da contagem e quantidades informadas pelo contador.

Essa persistência local permite que o aplicativo mantenha os dados organizados no próprio dispositivo. Antes da sincronização com a API REST, as quantidades registradas pelo contador são preservadas localmente, reduzindo o risco de perda das informações durante o processo de contagem.

As principais informações armazenadas localmente são:

Contagens disponíveis para o contador.

Produtos vinculados às contagens.

Itens da contagem.

Quantidades informadas pelo contador.

Status de sincronização com a API.

10.6 Integração com a API REST

A integração entre o aplicativo mobile e o backend web é realizada por meio da API REST desenvolvida no Laravel. O aplicativo consome rotas da API para autenticar usuários, consultar produtos, acessar o resumo mobile, listar contagens disponíveis, consultar os itens vinculados a uma contagem e sincronizar as quantidades informadas pelo contador.

As informações sincronizadas pelo aplicativo são processadas pelo back-end do sistema, armazenadas no banco de dados MySQL e posteriormente exibidas na aplicação web. Dessa forma, o administrador pode analisar as divergências entre o estoque registrado e o estoque contado.

A comunicação entre o aplicativo mobile e a API é realizada com dados em formato JSON, permitindo uma troca de informações estruturada e compatível entre as tecnologias utilizadas.

10.7 Benefícios do aplicativo mobile

O aplicativo mobile agrega valor ao projeto por facilitar a etapa prática de contagem de estoque. Ele permite que o contador registre as quantidades diretamente no local onde os produtos estão armazenados, evitando retrabalho e reduzindo a dependência de registros manuais.

Entre os principais benefícios do aplicativo mobile, destacam-se:

Maior praticidade na contagem física dos produtos.

Redução de erros causados por anotações manuais.

Sincronização dos dados com o sistema web por meio da API REST.

Armazenamento local dos dados com Room Database.

Melhor organização das informações coletadas durante a contagem.

Facilidade para posterior análise das divergências no sistema web.

Com isso, o aplicativo mobile complementa a aplicação web e torna o Counter uma solução mais completa para controle e contagem de estoque.

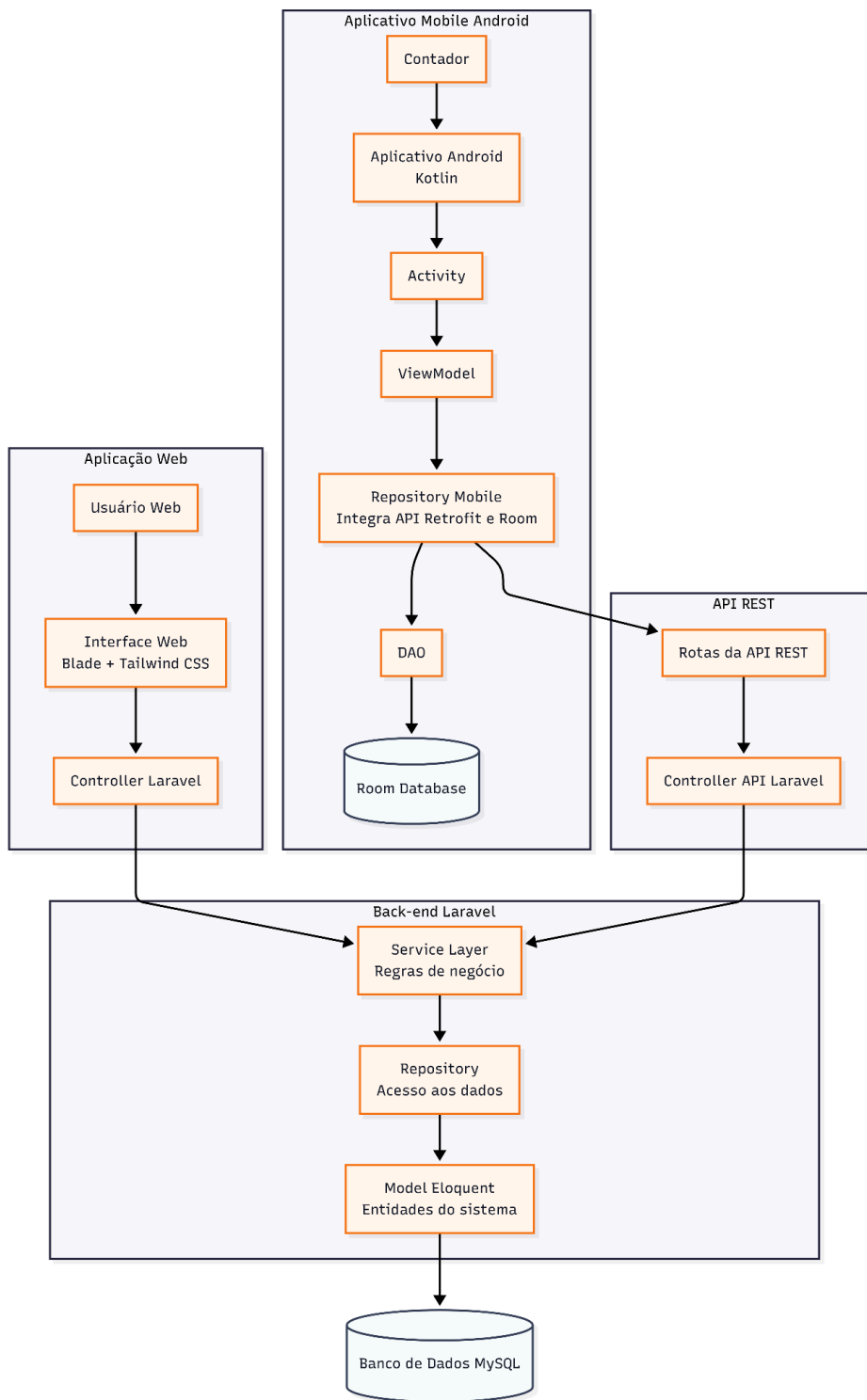
11. DIAGRAMAS

Os diagramas apresentados nesta seção têm como objetivo facilitar a compreensão da estrutura do sistema Counter, demonstrando de forma visual a arquitetura geral da aplicação, a organização do banco de dados e o fluxo principal da contagem de estoque.

11.1 Diagrama de arquitetura

A arquitetura geral do Counter é composta pela aplicação web, API REST, banco de dados MySQL e aplicativo mobile Android. A aplicação web é utilizada para o gerenciamento administrativo do estoque, enquanto o aplicativo mobile é utilizado pelo contador para registrar as quantidades físicas encontradas durante a contagem.

A estrutura simplificada da arquitetura pode ser visualizada no diagrama a seguir:



Esse diagrama demonstra a separação entre interface, regras de negócio, acesso a dados e persistência, tanto na aplicação web quanto no aplicativo mobile. Na aplicação web, o fluxo passa pela interface, Controller, Service, Repository, Model e banco de dados MySQL. Já no aplicativo mobile, a estrutura é composta por Activity, ViewModel, Repository, DAO e Room Database, com integração ao back-end por meio da API REST.

11.2 Diagrama do banco de dados

O banco de dados do Counter é composto por tabelas relacionadas entre si, permitindo organizar empresas, usuários, produtos, fornecedores, categorias, movimentações e contagens de estoque.

Os principais relacionamentos do banco são:

Uma empresa pode possuir vários usuários.

Uma empresa pode possuir várias categorias.

Uma empresa pode possuir vários fornecedores.

Uma empresa pode possuir vários produtos.

Uma categoria pode possuir vários produtos.

Um fornecedor pode possuir vários produtos.

Um produto pode possuir várias movimentações de estoque.

Um usuário pode registrar várias movimentações de estoque.

Uma empresa pode possuir várias contagens de estoque.

Uma contagem de estoque pode possuir vários itens contados.

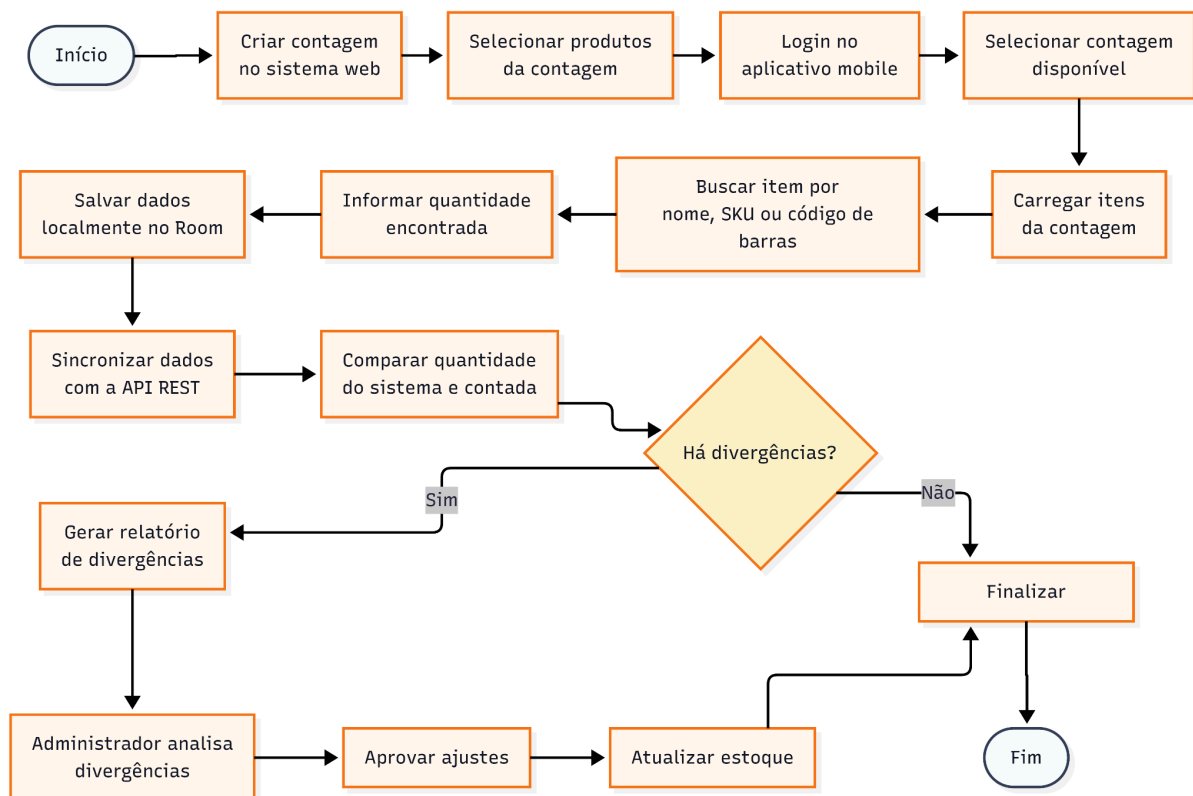
Um produto pode estar presente em vários itens de contagem.

Essa estrutura permite que o sistema mantenha os dados organizados e relacionados corretamente, garantindo a rastreabilidade das operações realizadas no estoque.

11.3 Diagrama do fluxo de contagem

O fluxo de contagem de estoque representa uma das principais funcionalidades do Counter. Esse processo envolve a criação de uma contagem no sistema web, a realização da conferência pelo aplicativo mobile e a análise das divergências pelo administrador.

O fluxo pode ser visualizado no diagrama a seguir:



O processo inicia quando o administrador cria uma contagem de estoque no sistema web. Em seguida, o contador acessa o aplicativo mobile, seleciona a contagem disponível, visualiza os produtos relacionados e informa as quantidades físicas encontradas. Os dados são salvos localmente no dispositivo e enviados para a API REST, que realiza a integração com o sistema web.

Após o envio, o sistema compara a quantidade registrada no estoque com a quantidade informada na contagem. Quando existem divergências, o sistema gera o relatório correspondente para análise do administrador. Após a aprovação dos ajustes, o estoque é atualizado. Caso não existam divergências, a contagem pode ser finalizada sem necessidade de correção.

Esse fluxo demonstra a integração entre a aplicação web, o aplicativo mobile, a API REST e o banco de dados. Além disso, evidencia como o processo de contagem é realizado de maneira controlada, evitando alterações automáticas sem a aprovação do administrador.

11.4 Importância dos diagramas

Os diagramas auxiliam na compreensão da estrutura geral do projeto e tornam mais clara a divisão de responsabilidades entre as partes do sistema. Por meio deles, é possível visualizar como os usuários interagem com a aplicação, como os dados são processados pelas camadas do sistema e como as informações são armazenadas no banco de dados.

No contexto do Counter, os diagramas também ajudam a demonstrar a aplicação prática da arquitetura em camadas, da integração com API REST e da comunicação entre o sistema web e o aplicativo mobile.

12. PRINTS DA APLICAÇÃO

Esta seção apresenta as principais telas desenvolvidas no projeto Counter. Os prints têm como objetivo demonstrar visualmente o funcionamento da aplicação web e do aplicativo mobile, evidenciando as funcionalidades implementadas, a organização da interface e a navegação do sistema.

Cada print é acompanhado por uma breve descrição, explicando a finalidade da tela e sua importância dentro do fluxo do sistema.

12.1 Tela de login web

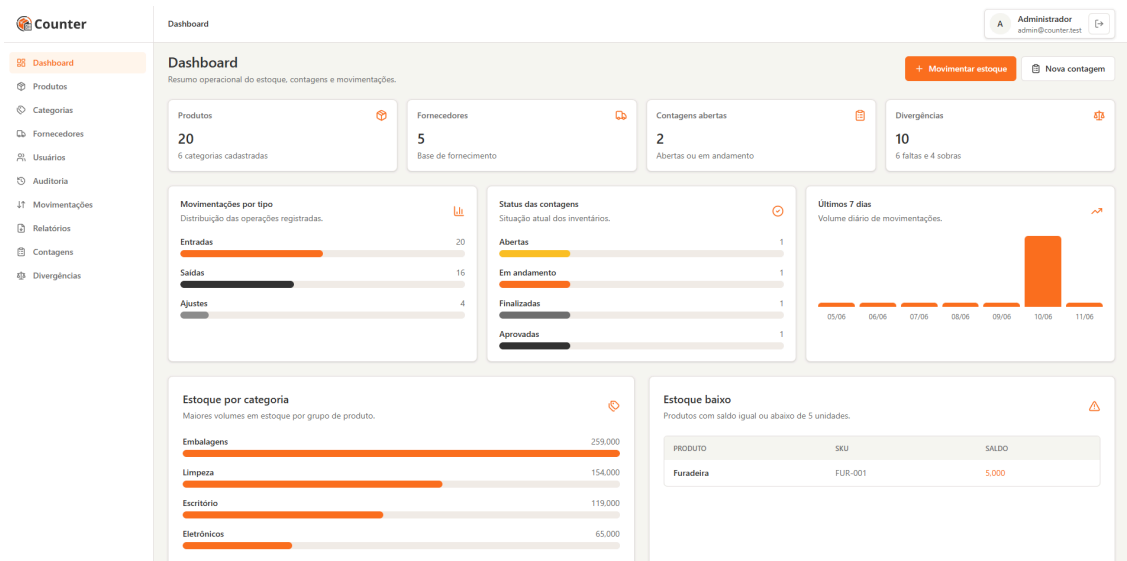


O print mostra a interface de login do sistema Counter. No topo, há o logotipo do Counter, composto por um ícone de uma caixa com um ponto de exclamação e o nome "Counter". Abaixo, o título "Entrar" é seguido por uma instrução: "Informe suas credenciais para acessar o sistema.".

Existem dois campos de entrada: "E-mail" com o valor "exemplo@empresa.com" e "Senha". O campo de senha possui um ícone de olho para alternar a visibilidade. Abaixo dos campos, há uma opção "Manter conectado" com uma caixa de seleção desmarcada. No final, há um botão laranja com o texto "Entrar".

A tela de login web permite que os usuários acessem o sistema informando suas credenciais, garantindo que apenas pessoas autenticadas utilizem as funcionalidades do Counter.

12.2 Dashboard web



O dashboard web apresenta uma visão geral do sistema, exibindo informações como produtos cadastrados, categorias, fornecedores, movimentações recentes e contagens em andamento.

12.3 Produtos

The Produtos page allows users to manage the inventory items. It includes a sidebar with navigation links for Dashboard, Produtos, Categorias, Fornecedores, Usuários, Auditoria, Movimentações, Relatórios, Contagens, and Divergências. The main content area displays a table of products with the following columns: PRODUTO, SKU, CATEGORIA, FORNECEDOR, QUANTIDADE, PREÇO VENDA, and AÇÕES. The table lists 10 products, including Álcool 70%, Café torrado, Caixa grande, Caixa pequena, Caneta azul, Chave de fenda, Detergente neutro, Etiqueta adesiva, Fita adesiva, and Furadeira. Each product entry includes its SKU, category, supplier, quantity, and selling price. The interface also includes a search bar, a filter for 'Novo produto', and a pagination control at the bottom showing 'Mostrando 1 até 10 de 20 registros'.

PRODUTO	SKU	CATEGORIA	FORNECEDOR	QUANTIDADE	PREÇO VENDA	AÇÕES
Álcool 70% 789000000011	ALC-001	Limpeza	Limpa Tudo Comercial	35,000 un	R\$ 14,90	 
Café torrado 789000000020	CAF-001	Alimentos	Office Mais Atacado	32,000 pct	R\$ 24,90	 
Caixa grande 789000000018	CGG-001	Embalagens	Pack Box Soluções	75,000 un	R\$ 8,90	 
Caixa pequena 789000000017	CXP-001	Embalagens	Pack Box Soluções	120,000 un	R\$ 3,50	 
Caneta azul 789000000007	CAN-001	Escritório	Office Mais Atacado	45,000 cx	R\$ 49,90	 
Chave de fenda 789000000015	CHF-001	Ferramentas	Ferramentas Norte	20,000 un	R\$ 22,90	 
Detergente neutro 789000000010	DET-001	Limpeza	Limpa Tudo Comercial	80,000 un	R\$ 5,50	 
Etiqueta adesiva 789000000009	ETI-001	Escritório	Office Mais Atacado	28,000 pct	R\$ 34,90	 
Fita adesiva 789000000019	FIT-001	Embalagens	Pack Box Soluções	64,000 rolo	R\$ 11,90	 
Furadeira 789000000016	FUR-001	Ferramentas	Ferramentas Norte	5,000 un	R\$ 279,90	 

A tela de produtos permite visualizar e gerenciar os itens cadastrados no estoque, incluindo informações como nome, SKU, código de barras, unidade de medida, quantidade atual, categoria e fornecedor.

12.4 Categorias

Counter

Dashboard

Produtos

Categorias

Fornecedores

Usuários

Auditoria

Movimentações

Relatórios

Contagens

Divergências

Categorias

Administrador
admin@counter.test

+ Nova categoria

Categorias

Organize os produtos em grupos para facilitar consulta e controle.

NOME	DESCRIÇÃO	PRODUTOS	AÇÕES
Alimentos	Itens de consumo interno	1	 
Eletrônicos	Produtos eletrônicos e periféricos	5	 
Embalagens	Caixas, fitas e materiais de envio	3	 
Escritório	Materiais usados em ambiente administrativo	4	 
Ferramentas	Ferramentas manuais e acessórios	3	 
Limpeza	Produtos de higiene e limpeza	4	 

A tela de categorias permite organizar os produtos em grupos, facilitando a classificação, consulta e controle dos itens cadastrados no estoque.

12.5 Fornecedores

Counter

Dashboard

Produtos

Categorias

Fornecedores

Usuários

Auditoria

Movimentações

Relatórios

Contagens

Divergências

Fornecedores

Administrador
admin@counter.test

+ Novo fornecedor

Fornecedores

Cadastre e consulte fornecedores vinculados aos produtos do estoque.

NOME	CNPJ	CONTATO	PRODUTOS	AÇÕES
Ferramentas Norte	45.678.901/0001-30	(11) 3000-0003 vendas@ferramentasnorte.test	3	 
Limpa Tudo Comercial	34.567.890/0001-20	(11) 3000-0002 contato@limpatudo.test	4	 
Office Mais Atacado	23.456.789/0001-10	(11) 3000-0001 pedidos@officemais.test	5	 
Pack Box Soluções	56.789.012/0001-40	(11) 3000-0004 comercial@packbox.test	3	 
Tech Prime Distribuidora	12.345.678/0001-90	(11) 3000-0000 vendas@techprime.test	5	 

A tela de fornecedores permite cadastrar e consultar os fornecedores vinculados aos produtos, contribuindo para a organização e rastreabilidade dos itens do estoque.

12.6 Movimentações de estoque

Counter

Dashboard

Produtos

Categorias

Fornecedores

Usuários

Auditoria

Movimentações

Relatórios

Contagens

Divergências

Movimentações

Administrador
admin@counter.test

+ Nova movimentação

Movimentações
Acompanhe entradas, saídas e ajustes de estoque.

Produto

Tipo

Usuário

De

Até

Limpar

Filtrar

PRODUTO	TIPO	QUANTIDADE	SALDO ANTES	SALDO DEPOIS	USUÁRIO	DATA
Café torrado	Ajuste	32,000	30,000	32,000	Estoquista	10/06/2026 16:14
Luva descartável	Ajuste	25,000	22,000	25,000	Estoquista	10/06/2026 16:14
Etiqueta adesiva	Saída	3,000	31,000	28,000	Estoquista	10/06/2026 16:14
Teclado mecânico	Saída	2,000	14,000	12,000	Estoquista	10/06/2026 16:14
Papel A4	Ajuste	30,000	32,000	30,000	Estoquista	10/06/2026 16:14
Café torrado	Saída	8,000	40,000	32,000	Estoquista	10/06/2026 16:14
Café torrado	Entrada	40,000	0,000	40,000	Estoquista	10/06/2026 16:14
Fita adesiva	Saída	6,000	70,000	64,000	Estoquista	10/06/2026 16:14
Fita adesiva	Entrada	70,000	0,000	70,000	Estoquista	10/06/2026 16:14
Caixa grande	Saída	5,000	80,000	75,000	Estoquista	10/06/2026 16:14
Caixa grande	Entrada	80,000	0,000	80,000	Estoquista	10/06/2026 16:14
Caixa pequena	Saída	30,000	150,000	120,000	Estoquista	10/06/2026 16:14
Caixa pequena	Entrada	150,000	0,000	150,000	Estoquista	10/06/2026 16:14

A tela de movimentações de estoque permite registrar e consultar entradas, saídas e ajustes de produtos, mantendo o histórico das alterações realizadas nas quantidades.

12.7 Contagens de estoque

Counter

Dashboard

Produtos

Categorias

Fornecedores

Usuários

Auditoria

Movimentações

Contagens

Divergências

Contagens

Administrador
admin@counter.test

+ Nova contagem

Contagens
Crie conferências de estoque e acompanhe os produtos vinculados.

TÍTULO	STATUS	PRODUTOS	CRIADA POR	INÍCIO	AÇÕES
Contagem limpeza aprovada	Aprovada	4	Administrador	30/05/2026 00:06	
Contagem escritório	Finalizada	4	Administrador	06/06/2026 00:06	
Contagem almoxarifado	Em andamento	4	Administrador	10/06/2026 18:06	
Contagem piloto	Em andamento	5	Administrador	09/06/2026 00:06	

A tela de contagens de estoque permite criar e acompanhar processos de conferência física dos produtos, exibindo informações como status e dados principais de cada contagem.

12.8 Itens da contagem

Counter

Dashboard

Produtos

Categorias

Fornecedores

Usuários

Auditoria

Movimentações

Relatórios

Contagens

Divergências

Contagem limpeza aprovada

Administrador
admin@counter.test

Aprovada

< Voltar para contagens

Contagem limpeza aprovada

Saldo congelado em 30/05/2026 00:06

Produtos

4

Criada por

Administrador

Status

Aprovada

PRODUTO	SKU	SALDO DO SISTEMA	QUANTIDADE CONTADA	DIFERENÇA	CONTADO POR	SINCRONIZAÇÃO
Detergente neutro	DET-001	82,000	80,000	-2,000	Contador	Sincronizada
Álcool 70%	ALC-001	34,000	35,000	1,000	Contador	Sincronizada
Luva descartável	LUV-001	25,000	25,000	0,000	Contador	Sincronizada
Martelo	MAR-001	9,000	8,000	-1,000	Contador	Sincronizada

Voltar

Salvar contagem

Ajustes aprovados

Movimentações de ajuste geradas pela aprovação desta contagem.

Nenhum ajuste foi necessário.

Aprovação de ajustes

Finalize a contagem e aprove os ajustes para atualizar o estoque com as quantidades contadas.

A tela de itens da contagem apresenta os produtos vinculados a uma contagem específica, permitindo visualizar as informações necessárias para a conferência do estoque.

12.9 Divergências

Counter

Dashboard

Produtos

Categorias

Fornecedores

Usuários

Auditoria

Movimentações

Relatórios

Contagens

Divergências

Divergências

Administrador
admin@counter.test

Divergências

Compare o saldo do sistema com as quantidades contadas fisicamente.

Falta física

6

Sobra física

4

Sem divergência

3

Tipo

Todos

Limpar

Filtrar

PRODUTO	CONTAGEM	SISTEMA	CONTADO	DIFERENÇA	TIPO	AÇÕES
Caneta azul SKU CAN-001	Contagem piloto	45,000	42,000	-3,000	Falta física	
Caixa pequena SKU CXP-001	Contagem piloto	120,000	126,000	6,000	Sobra física	
Pasta suspensa SKU PAS-001	Contagem escritório	16,000	15,000	-1,000	Falta física	
Etiqueta adesiva SKU ETI-001	Contagem escritório	28,000	31,000	3,000	Sobra física	
Café torrado SKU CAF-001	Contagem escritório	32,000	32,000	0,000	Sem divergência	
Papel toalha SKU PTO-001	Contagem escritório	14,000	12,000	-2,000	Falta física	
Detergente neutro SKU DET-001	Contagem limpeza aprovada	82,000	80,000	-2,000	Falta física	
Álcool 70% SKU ALC-001	Contagem limpeza aprovada	34,000	35,000	1,000	Sobra física	

A tela de divergências apresenta as diferenças entre a quantidade registrada no sistema e a quantidade informada durante a contagem física dos produtos.

12.10 Relatórios

Counter

Dashboard

Produtos

Categorias

Fornecedores

Usuários

Auditoria

Movimentações

Relatórios

Contagens

Divergências

Relatórios

Administrador
admin@counter.test

Relatórios

Exporte dados de estoque, movimentações e divergências em CSV.

Produtos cadastrados

20

663.000 unidades em estoque

Valor de custo

R\$ 33.680,50

Baseado no saldo atual

Movimentações

40

20 entradas, 16 saídas e 4 ajustes

Estoque atual

Produtos com SKU, categoria, fornecedor, unidade, quantidade e preços.

Exportar estoque

Movimentações

Entradas, saídas, ajustes, responsáveis, motivos e contagens vinculadas.

Produto

Todos

Tipo

Todos

Usuário

Todos

De

dd/mm/aaaa

Até

dd/mm/aaaa

Exportar movimentações

Divergências

Itens contados com falta física, sobra física ou sem divergência.

Faltas físicas

7

Sobras físicas

6

Sem divergência

3

Todos

Exportar divergências

A tela de relatórios permite visualizar informações consolidadas sobre estoque, movimentações, contagens e divergências, auxiliando na análise dos dados do sistema.

12.11 Auditoria

Counter

Dashboard

Produtos

Categorias

Fornecedores

Usuários

Auditoria

Movimentações

Relatórios

Contagens

Divergências

Auditoria

Administrador
admin@counter.test

Auditoria

Acompanhe ações administrativas e operações importantes do estoque.

Módulo

Todos

Ação

Todas

Usuário

Todos

De

dd/mm/aaaa

Até

dd/mm/aaaa

Limpar

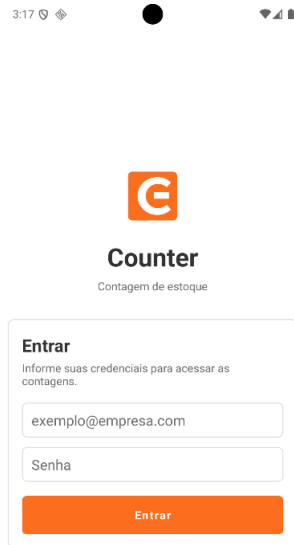
Filtrar

DATA	USUÁRIO	MÓDULO	AÇÃO	DESCRIÇÃO	ORIGEM
13/06/2026 02:30	Administrador admin@counter.test	Usuários	Atualizou	Usuário atualizado: Administrador	127.0.0.1
11/06/2026 19:11	Contador contador@counter.test	Contagens	Alterou	Itens da contagem atualizados: Contagem almoxarifado	127.0.0.1
11/06/2026 19:03	Contador contador@counter.test	Contagens	Alterou	Itens da contagem atualizados: Contagem almoxarifado	127.0.0.1
11/06/2026 18:41	Contador contador@counter.test	Contagens	Alterou	Itens da contagem atualizados: Contagem almoxarifado	127.0.0.1
11/06/2026 18:37	Contador contador@counter.test	Contagens	Alterou	Itens da contagem atualizados: Contagem piloto	127.0.0.1
11/06/2026 17:56	Contador contador@counter.test	Contagens	Alterou	Itens da contagem atualizados: Contagem almoxarifado	127.0.0.1
10/06/2026 18:32	Administrador admin@counter.test	Produtos	Atualizou	Produto atualizado: Caixa grande	127.0.0.1
10/06/2026 18:27	Administrador admin@counter.test	Produtos	Atualizou	Produto atualizado: Álcool 70%	127.0.0.1
10/06/2026 18:26	Administrador admin@counter.test	Usuários	Atualizou	Usuário atualizado: Administrador	127.0.0.1

A tela de auditoria apresenta registros de operações importantes realizadas no sistema, permitindo acompanhar ações executadas pelos usuários.

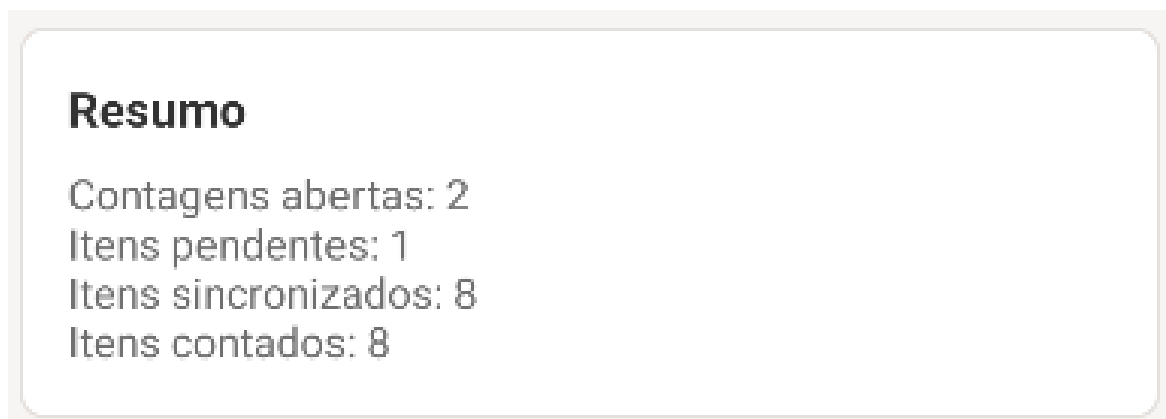
40

12.12 Login mobile



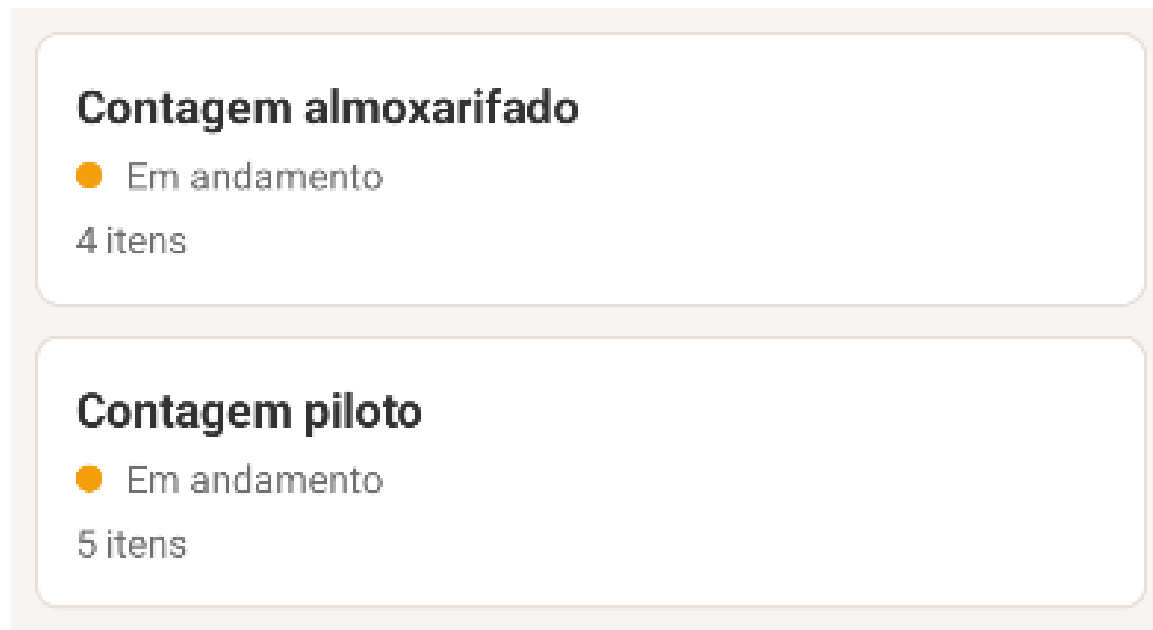
A tela de login mobile permite que o contador acesse o aplicativo Android com suas credenciais para realizar as atividades de contagem de estoque.

12.13 Resumo mobile



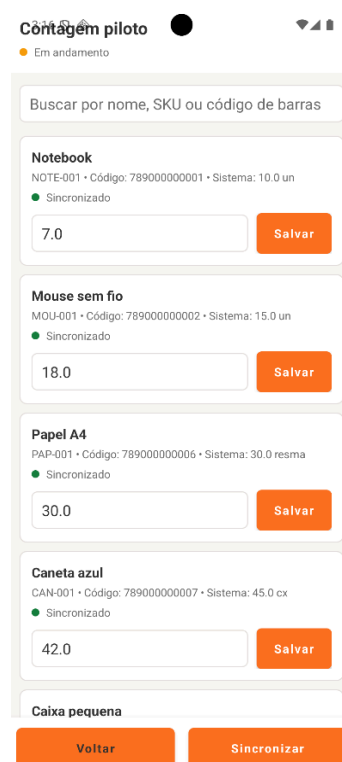
A tela de resumo mobile apresenta informações gerais para o contador, como contagens disponíveis, itens pendentes e status de sincronização.

12.14 Contagens disponíveis no mobile



A tela de contagens disponíveis no mobile permite que o contador visualize e selecione a contagem de estoque que será realizada pelo aplicativo.

12.15 Itens da contagem no mobile



A tela de itens da contagem no mobile permite que o contador visualize os produtos vinculados à contagem, consulte informações como nome, SKU, código de barras, unidade de medida, quantidade do sistema e status de sincronização, utilize busca textual para localizar itens e informe as quantidades encontradas para sincronização com a API REST.

13. DIFICULDADES ENCONTRADAS

Durante o desenvolvimento do projeto Counter, algumas dificuldades foram identificadas em razão da necessidade de construir uma solução web completa, integrada a um aplicativo mobile e estruturada com boas práticas de desenvolvimento de software.

Uma das principais dificuldades foi a definição da arquitetura do sistema. Como o projeto envolve aplicação web, API REST, banco de dados e aplicativo mobile, foi necessário organizar corretamente as responsabilidades de cada parte da solução. Para isso, foi adotada uma arquitetura baseada em MVC com camadas adicionais de Service, Repository e DTO, evitando que as regras de negócio ficassem concentradas diretamente nos Controllers.

Outra dificuldade encontrada foi a aplicação dos Design Patterns de forma coerente dentro do projeto. O desafio não foi apenas utilizar padrões de projeto, mas aplicá-los em pontos onde realmente fizessem sentido. Por esse motivo, foram escolhidos padrões relacionados às necessidades do sistema, como Repository para acesso ao banco de dados, Service Layer para regras de negócio, DTO para transporte de dados, Dependency Injection para reduzir acoplamento entre classes e Facade para simplificar o uso de transações do banco.

A modelagem do banco de dados também exigiu atenção, principalmente na definição das tabelas relacionadas ao processo de contagem de estoque. Foi necessário pensar em uma estrutura que permitisse armazenar os produtos cadastrados, as movimentações realizadas, as contagens abertas e os itens contados. Além disso, foi necessário definir relacionamentos entre empresas, usuários, produtos, categorias, fornecedores, movimentações e contagens.

A integração entre o sistema web e o aplicativo mobile também representou uma dificuldade importante. Como o aplicativo Android consome dados do backend Laravel, foi necessário planejar uma API REST com rotas organizadas, autenticação, retornos em JSON, validações e tratamento de erros. Essa integração é essencial para que o contador consiga realizar a contagem pelo celular e sincronizar as quantidades informadas com o sistema web.

No aplicativo mobile, uma das dificuldades foi compreender e aplicar a estrutura utilizando Kotlin, ViewModel, Repository, DAO e Room Database. Essa organização exige a separação entre interface, lógica de apresentação, acesso aos dados e persistência local. Apesar de tornar o aplicativo mais organizado, essa estrutura também aumentou a complexidade inicial do desenvolvimento.

Também foi necessário definir corretamente o fluxo de contagem de estoque. O sistema precisa permitir que o administrador crie uma contagem, que o contador informe as quantidades físicas pelo aplicativo mobile e que o administrador visualize

as divergências antes de aprovar os ajustes. Esse fluxo exigiu cuidado para evitar que o estoque fosse alterado automaticamente sem conferência.

Outra dificuldade foi manter o projeto simples o suficiente para ser desenvolvido dentro do prazo, mas completo o bastante para atender aos requisitos da atividade. Por isso, algumas funcionalidades foram deixadas fora do escopo inicial, como alerta de estoque mínimo, emissão de notas fiscais, controle financeiro e integração com sistemas externos.

Apesar das dificuldades encontradas, elas contribuíram para melhorar o planejamento do sistema e reforçar a importância da organização do código, da separação de responsabilidades, da modelagem correta do banco de dados e da integração entre diferentes partes de uma aplicação.

14. CONCLUSÃO

O desenvolvimento do projeto Counter permitiu aplicar, de forma prática, os principais conceitos estudados na disciplina de Programação III. A proposta do sistema foi criar uma solução web e mobile para controle e contagem de estoque, contemplando funcionalidades essenciais como cadastro de produtos, categorias e fornecedores, registro de entradas e saídas, histórico de movimentações, criação de contagens físicas, análise de divergências e integração com aplicativo mobile.

A aplicação foi desenvolvida com uma arquitetura organizada, baseada no padrão MVC do Laravel e complementada por camadas de Service, Repository e DTO. Essa estrutura contribui para a separação de responsabilidades, tornando o código mais limpo, modular e de melhor manutenção. Além disso, a utilização de Design Patterns como Repository Pattern, Service Layer, DTO, Dependency Injection e Facade reforça a aplicação de boas práticas de desenvolvimento de software.

O uso do banco de dados MySQL possibilita a persistência das informações do sistema, permitindo armazenar usuários, empresas, produtos, fornecedores, categorias, movimentações e contagens de estoque. A modelagem relacional contribui para a integridade dos dados e para o correto funcionamento das regras de negócio definidas no projeto.

A API REST representa uma parte importante da solução, pois permite a comunicação entre o aplicativo mobile Android e o backend web. Por meio dessa integração, o contador realiza a conferência física dos produtos pelo celular, informa as quantidades encontradas e sincroniza os dados com o sistema, enquanto a aplicação web concentra a administração geral, a análise das divergências e a aprovação dos ajustes necessários.

O aplicativo mobile, desenvolvido em Kotlin com uso de Room Database, complementa o sistema ao oferecer uma forma prática de registrar as quantidades

físicas encontradas no estoque. Essa funcionalidade aproxima o projeto de uma situação real de uso, em que a contagem de produtos normalmente ocorre diretamente no ambiente físico da empresa.

Portanto, o Counter atende ao objetivo proposto de desenvolver uma solução completa, funcional e organizada para controle de estoque. O projeto demonstra a integração entre front-end web, back-end, banco de dados, API REST, aplicativo mobile, arquitetura em camadas e Design Patterns, resultando em uma aplicação coerente com os requisitos técnicos definidos para a atividade.

15. REFERÊNCIAS

LARAVEL. Laravel Documentation. Documentação oficial do framework Laravel. Disponível em: <https://laravel.com/docs>. Acesso em: 09 jun. 2026.

MYSQL. MySQL Documentation. Documentação oficial do banco de dados MySQL. Disponível em: <https://dev.mysql.com/doc/>. Acesso em: 09 jun. 2026.

ANDROID DEVELOPERS. Save data in a local database using Room. Documentação oficial do Room Database. Disponível em: <https://developer.android.com/training/data-storage/room>. Acesso em: 09 jun. 2026.

ANDROID DEVELOPERS. Kotlin and Android. Documentação oficial sobre desenvolvimento Android com Kotlin. Disponível em: <https://developer.android.com/kotlin>. Acesso em: 09 jun. 2026.

TAILWIND CSS. Tailwind CSS Documentation. Documentação oficial do framework Tailwind CSS. Disponível em: <https://tailwindcss.com/docs>. Acesso em: 09 jun. 2026.

ALPINE.JS. Alpine.js Documentation. Documentação oficial da biblioteca Alpine.js. Disponível em: <https://alpinejs.dev/>. Acesso em: 09 jun. 2026.

LUCIDE. Lucide Icons Documentation. Documentação oficial da biblioteca Lucide Icons. Disponível em: <https://lucide.dev/>. Acesso em: 09 jun. 2026.

REFactoring.GURU. Design Patterns. Material de apoio sobre padrões de projeto. Disponível em: <https://refactoring.guru/design-patterns>. Acesso em: 09 jun. 2026.

GITHUB. Repositório do projeto Counter. Disponível em: <https://github.com/tawamark/counter-app>. Acesso em: 12 jun. 2026.